

透過 Cloud Run，三個簡單步驟即可將開發環境部署至正式環境

來源：<https://codelabs.developers.google.com/codelabs/cloud-run-dev2prod?hl=zh-tw>

步驟數：11

在本程式碼研究室中，您將建構簡單的網頁應用程式，在私人開發人員環境中執行，然後使用 Docker 在容器中執行，最後在雲端部署相同的應用程式。過程中，您將瞭解容器是什麼、如何解決一些重要問題，以及如何透過幾個簡單步驟，從測試本機應用程式到部署可擴充、具備復原能力的高可用性版本，完全不必變更任何程式碼！

目錄

1. [1. 簡介](#)
2. [2. 進行設定](#)
3. [3. 建構簡單的網頁應用程式](#)
4. [4. 將應用程式容器化](#)
5. [5. 雲端...](#)
6. [6. 自動建立容器](#)
7. [7. 我是否必須使用指令列？](#)
8. [8. 測試應用程式](#)
9. [9. 清除](#)
10. [10. 你辦到了！](#)
11. [11. 行動號召](#)

1. 簡介

為什麼應用程式管理這麼困難？

其中一個主要原因是開發人員通常必須兼任系統管理員。請參考以下 (部分) 疑慮清單，以便開發、部署及管理現代化生產環境等級的網頁應用程式：

**Things I
don't want
to think
about...**

1. Provisioning hardware
2. Installing software
3. Upgrading operating systems
4. Security patching
5. System and network admin
6. Scaling up/down
7. Paying for stuff I don't use
8. Dealing with failures
9. Managing clusters

我不知道你感覺如何，但這些都是我不想煩惱的事！我真正想思考的是應用程式邏輯：

**Things I want
to think
about...**

1. Solving my problem

簡而言之，這就是 Cloud Run 的核心價值：讓您專注於應用程式，將所有管理和維護工作交給 Google。Google 在這個領域投入了數百萬小時，不斷精進和完善相關技能。

除了上述管理挑戰外，您還必須處理：

- **依附元件** - 應用程式的執行環境應盡可能與測試環境完全一致。這可能涵蓋多個層面，包括作業系統、支援程式庫、語言解譯器或編譯器、硬體設定，以及許多其他因素。
- **發布** - 從應用程式的本機版本移至網路上廣泛分享的版本，通常需要變更執行階段環境、大幅提升複雜度，以及陡峭的學習曲線。

Cloud Run 會為您處理這些問題，以及許多其他問題。與其聽我說，不如**一起建構應用程式**，看看只要幾個簡單步驟，就能輕鬆從本機開發環境轉換為實際工作環境級別的雲端應用程式。

執行步驟

- 您將建構簡單的網頁應用程式，並確認該應用程式在開發環境中如預期執行。
- 接著，您會將相同應用程式移至容器化版本。過程中，您將瞭解容器化代表的意義，以及容器化為何如此實用。
- 最後，您將應用程式部署至雲端，並瞭解如何使用指令列和 Google Cloud 控制台輕鬆管理 Cloud Run 服務。

課程內容...

- 如何使用 Python 建立簡易的網路伺服器應用程式
- 如何將應用程式封裝至可在任何位置執行的 Docker 容器
- 如何將應用程式部署至雲端，讓任何人都能試用您的新作品

- 如何使用建構包進一步簡化上述步驟
- 如何使用 Google Cloud 指令列工具和 Cloud Console 網頁版 UI

事前準備

- 網路瀏覽器
- Google 帳戶

本實驗室適合各種程度的開發人員，包括初學者。雖然您會使用 Python，但不需要熟悉 Python 程式設計，就能瞭解發生了什麼事，因為我們會說明您使用的所有程式碼。

步驟 2 · 約 6 分鐘

2. 進行設定



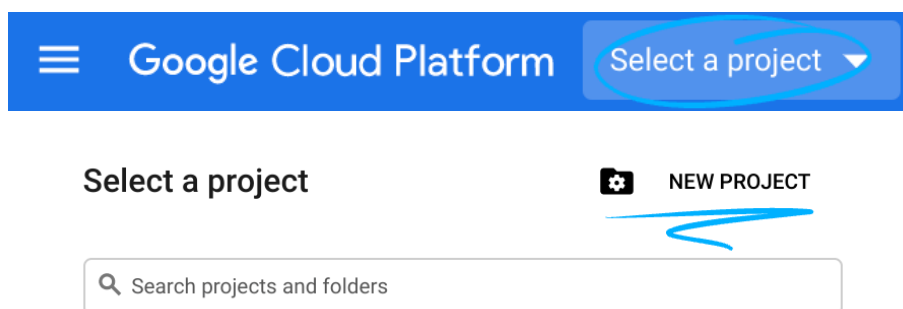
Photo by Danielle MacInnes on Unsplash

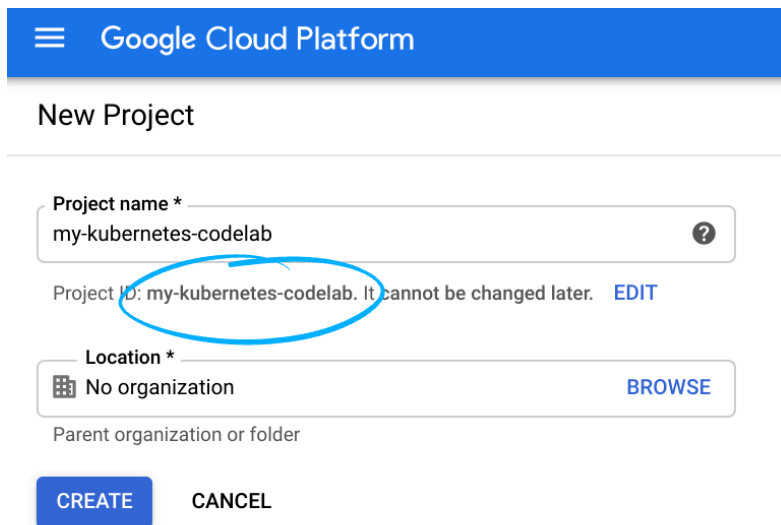
本節說明如何開始進行這個實驗室。

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。(如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#))。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：`console.cloud.google.com`。





Google Cloud Platform

New Project

Project name *
my-kubernetes-codelab

Project ID: my-kubernetes-codelab. It cannot be changed later. [EDIT](#)

Location *
No organization [BROWSE](#)

Parent organization or folder

[CREATE](#) [CANCEL](#)

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

在本實驗室中，您將在 [Cloud Shell](#) 工作階段中操作，這是由 Google Cloud 中執行的虛擬機器代管的指令解譯器。您也可以在本機電腦上輕鬆執行本節內容，但使用 Cloud Shell 可讓所有人在一致的環境中，獲得可重現的體驗。完成實驗室後，歡迎在自己的電腦上重試這個部分。

首次在 Cloud Shell 中發出需要授權的要求時，您會看到「授權 Cloud Shell」對話方塊 (如下所示)，請放心點選「授權」按鈕。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

[Authorize](#) [Reject](#)

啟用 Cloud Shell

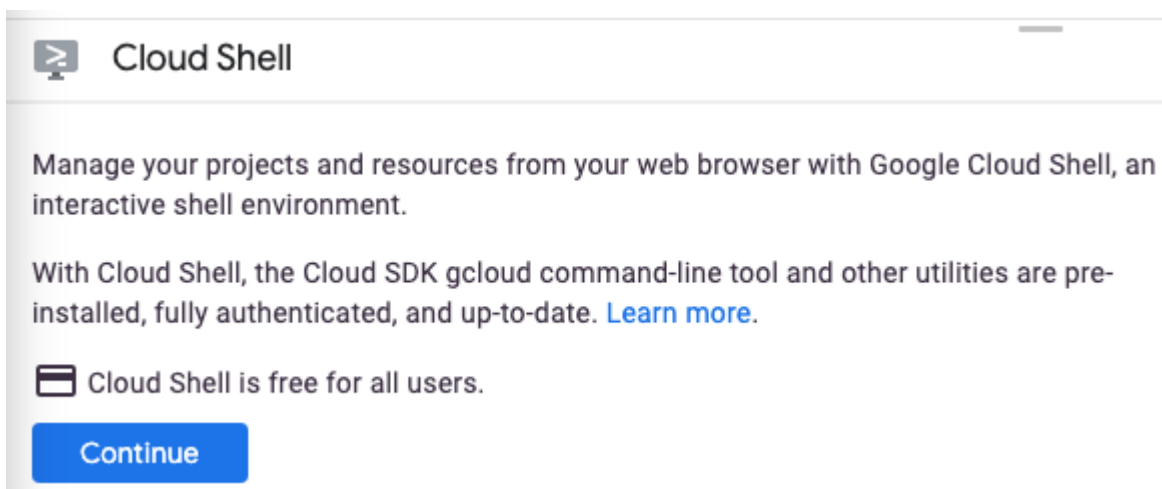
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



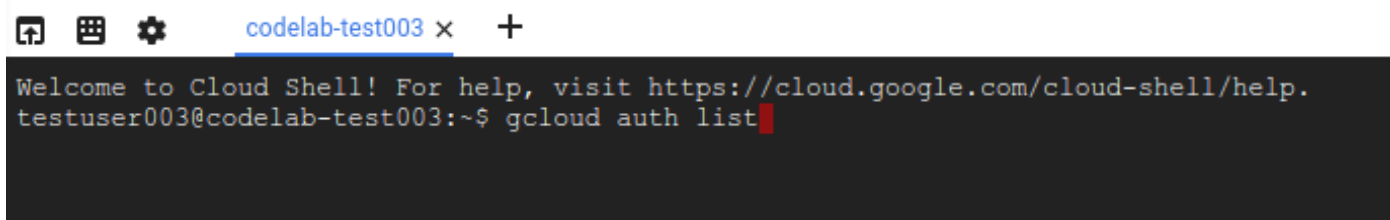
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (位於摺疊式選單下方)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器搭載您需要的所有開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

在終端機中設定一些環境變數，方便後續步驟操作：

```
export PROJ=$GOOGLE_CLOUD_PROJECT
export APP=hello
export PORT=8080
export REGION="us-central1"
export TAG="gcr.io/$PROJ/$APP"
```

啟用 API

在後續步驟中，您會瞭解為何需要這些服務，但現在請先執行下列指令，授予專案 Cloud Build、Container Registry 和 Cloud Run 服務的存取權：

```
gcloud services enable cloudbuild.googleapis.com \
                      containerregistry.googleapis.com \
                      run.googleapis.com
```

這時應該會顯示類似以下的成功訊息：

```
Operation "operations/acf.cc11852d-40af-47ad-9d59-477a12847c9e" finished
successfully.
```

步驟 3 · 約 6 分鐘

3. 建構簡單的網頁應用程式

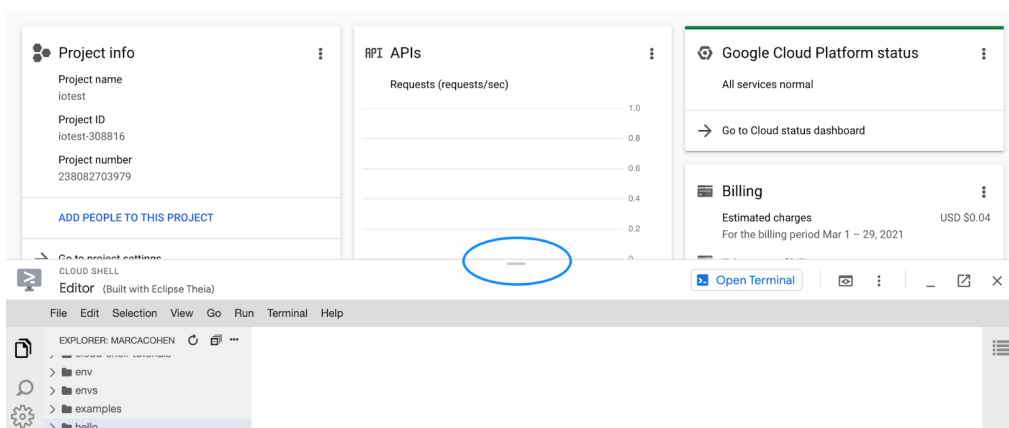


Photo by Trà My on Unsplash

首先，請點選 Cloud Shell 面板頂端的 `Open Editor` 按鈕。這是訂閱按鈕的圖示：



接著，您會進入類似 Visual Studio Code 的 IDE 環境，可以在其中建立專案、編輯原始碼、執行程式等。如果螢幕空間不足，可以拖曳控制台和編輯/終端機視窗之間的水平線 (如下圖所示)，展開或縮小這兩個區域：



如要在編輯器和終端機之間切換，請分別點選 `Open Editor` 和 `Open Terminal` 按鈕。現在請嘗試在這兩種環境之間來回切換。

此外，還有「在新視窗中開啟」按鈕 (如下所示：



)，可將 Cloud 控制台和終端機/編輯器分別顯示在不同視窗中。本實驗室不需要使用該功能，但瞭解這項功能很有幫助。

接著，選取「File」>「New Folder」，輸入 `hello`，然後按一下 `OK`，建立資料夾來儲存本實驗室的工作。您在本實驗室中建立的所有檔案，以及在 Cloud Shell 中執行的所有工作，都會在這個資料夾中進行。

現在請建立 `requirements.txt` 檔案。這會告知 Python 應用程式依附的程式庫。對於這個簡單的網頁應用程式，您將使用熱門的 Python 模組 (用於建構名為 `Flask` 的網路伺服器) 和名為 `gunicorn` 的網路伺服器架構。在 Cloud Editor 視窗中，按一下「File」->「New File」選單，即可建立新檔案。系統提示輸入新檔案名稱時，請輸入

`requirements.txt`，然後按下 `OK` 鍵。確認新檔案最終會出現在 `hello` 專案資料夾中。

在新檔案中輸入下列幾行，指定應用程式依附於 Python `Flask` 套件和 `gunicorn` 網頁伺服器。

```
Flask
gunicorn
```

您不必明確儲存這個檔案，因為 Cloud Editor 會自動儲存變更。

版本 1：Hello world!

使用相同技巧，建立另一個名為 `main.py` 的新檔案。這會是應用程式的主要 (也是唯一) Python 來源檔案。同樣地，請確認新檔案最終會位於 `hello` 專案資料夾中。

將下列程式碼插入這個檔案：

```
from flask import Flask
import os
import random

app = Flask(__name__) # Create a Flask object.
PORT = os.environ.get("PORT") # Get PORT setting from the environment.

# The app.route decorator routes any GET requests sent to the root path
# to this function, which responds with a "Hello world!" HTML document.
@app.route("/", methods=["GET"])
def say_hello():
    html = "<h1>Hello world!</h1>"
    return html

# This code ensures that your Flask app is started and listens for
# incoming connections on the local interface and port 8080.
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORT)
```

請詳閱行內註解 (「#」字元後的文字)，瞭解每行 Python 程式碼的作用。

切換回終端機，並使用下列指令切換至專案資料夾：

```
cd hello
```

如果發現 Cloud Shell 無法正常運作，可能是因為系統重新啟動，並返回主目錄。輸入 `pwd` (列印工作目錄) 即可查看目前是否位於主目錄。如果是，請執行 `cd hello` 指令，切換至專案資料夾。

執行下列指令來安裝專案依附元件：

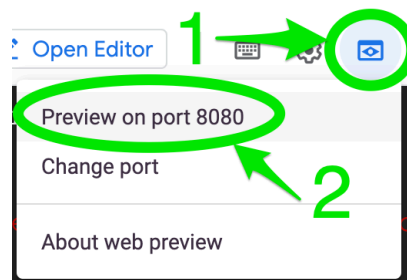
```
pip3 install -r requirements.txt
```

現在在終端機中執行下列指令，即可啟動應用程式：

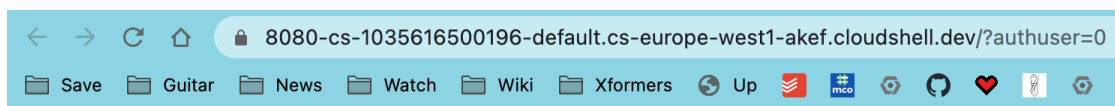
```
python3 main.py
```

此時，您的應用程式會在專為 Cloud Shell 工作階段建立的虛擬機器上執行。Cloud Shell 內含 Proxy 機制，可讓您從全球網際網路的任何位置，存取虛擬機器上執行的網路伺服器 (例如您剛啟動的伺服器)。

依序點選 `web preview` 按鈕和 `Preview on Port 8080` 選單項目，如下所示：



系統會開啟網路瀏覽器分頁，顯示執行中的應用程式，畫面應如下所示：



Hello world!

第 2 版：回應網址路徑

返回 Cloud Editor (透過 `Open Editor` 按鈕)，並更新 `main.py` 檔案，新增支援回傳選用網址尾碼的功能，如下所示：

請注意標有 NEW 和 MODIFIED 註解的行。以下是與前一版相比的異動。您可以選擇性地進行這些變更，也可以將整個 `main.py` 內容替換為下列程式碼。


```

from flask import Flask
import os
import random

app = Flask(__name__) # Create a Flask object.
PORT = os.environ.get("PORT") # Get PORT setting from environment.

# The app.route decorator routes any GET requests sent to the root path
# to this function, which responds with a "Hello world!" HTML document.
# If something is specified as the URL path (after the '/'), say_hello()
# responds with "Hello X", where X is the string at the end of the URL.
@app.route("/", methods=["GET"])
@app.route("/<name>", methods=["GET"]) # ← NEW
def say_hello(name="world"):          # ← MODIFIED
    html = f"<h1>Hello {name}!</h1>"  # ← MODIFIED
    return html

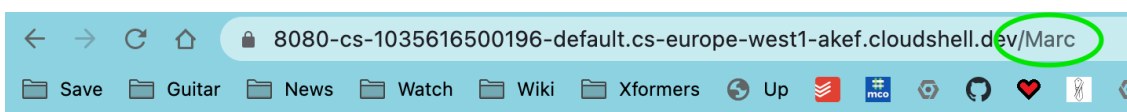
# This code ensures that your Flask app is started and listens for
# incoming connections on the local interface and port 8080.
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORT)

```

切換回終端機 (透過 `Open Terminal` 按鈕)，然後輸入 `control-C` (按住 `Ctrl` 鍵並按下「C」)，停止執行中的應用程式，然後輸入下列內容重新啟動應用程式：

```
python3 main.py
```

再次按一下 `web preview` 按鈕，然後按一下 `Preview on Port 8080` 選單項目，開啟網頁瀏覽器分頁，前往正在執行的應用程式。您應該會再次看到「Hello world!」訊息，但現在請將斜線字元後的網址文字取代為您選擇的任何字串 (例如 `/your-name`)，並確認您看到類似以下的內容：



Hello Marc!

版本 3：隨機顏色

現在，請返回 Cloud Editor (透過 `Open Editor` 按鈕)，並按照下列方式更新 `main.py` 檔案，新增隨機背景顏色支援功能：

請注意標有 `NEW` 和 `MODIFIED` 註解的行。以下是與前一版相比的異動。您可以選擇性地進行這些變更，也可以將整個 `main.py` 內容替換為下列程式碼。


```

from flask import Flask
import os
import random

app = Flask(__name__) # Create a Flask object.
PORT = os.environ.get("PORT") # Get PORT setting from the environment.

# This function decides whether foreground text should be
# displayed in black or white, to maximize fg/bg contrast.
def set_text_color(rgb):
    sum = round(
        (int(rgb[0]) * 0.299)
        + (int(rgb[1]) * 0.587)
        + (int(rgb[2]) * 0.114)
    )
    return "black" if sum > 186 else "white"

# The app.route decorator routes any GET requests sent to the root path
# to this function, which responds with a "Hello world!" HTML document.
# If something is specified as the URL path (after the '/'), say_hello()
# responds with "Hello X", where X is the string at the end of the URL.
# To verify each new invocation of these requests, the HTML document
# includes CSS styling to produce a randomly colored background.
@app.route("/", methods=["GET"])
@app.route("/<name>", methods=["GET"])
def say_hello(name="world"):
    bg = random.sample(range(1, 255), 3)
    hex = (int(bg[0]) * 256) + (int(bg[1]) * 16) + int(bg[2])
    fg_color = set_text_color(bg)
    bg_color = f"#{hex:06x}"
    style = f"color:{fg_color}; background-color:{bg_color}"
    html = f'<h1 style="{style}">Hello {name}!</h1>'
    return html

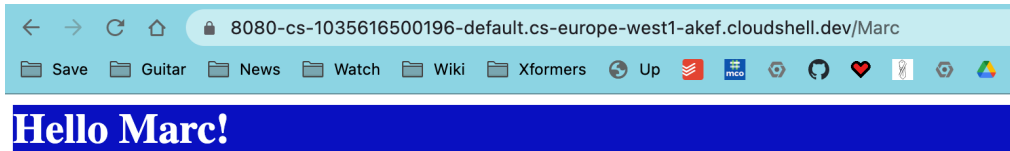
# This code ensures that your Flask app is started and listens for
# incoming connections on the local interface and port 8080.
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORT)

```

切換回終端機 (透過 `Open Terminal` 按鈕)，然後輸入 `control-C` (按住 `Ctrl` 鍵並按下「C」)，停止執行中的應用程式，然後輸入下列內容重新啟動應用程式：

```
python3 main.py
```

再次點選 `web preview` 按鈕，然後點選 `Preview on Port 8080` 選單項目，開啟執行中應用程式的網頁瀏覽器分頁。您應該會看到產生的文字，以及任何指定的後置字串或預設的「Hello world!」字串，顯示在隨機顏色的背景前方，如下所示：



重新載入頁面幾次，您會發現每次造訪應用程式時，背景顏色都會隨機變更。

這樣就完成應用程式了，恭喜！在下一個步驟中，您將瞭解如何將應用程式封裝到容器中，以及這麼做的好處。

4. 將應用程式容器化



Photo by Erda Estremera on Unsplash

什麼是容器？

一般容器 (尤其是 Docker) 可讓我們建立模組化方塊，在其中執行應用程式，並將所有依附元件一併封裝。我們將結果稱為容器映像檔。在本節中，您將建立容器映像檔，用於封裝應用程式及其所有依附元件。

說到依附元件，在先前的步驟中，您在開發人員環境中執行應用程式時，必須執行 `pip3 install -r requirements.txt`，並確保 `requirements.txt` 檔案包含所有依附程式庫和對應版本。使用容器時，您會在產生容器映像檔時安裝這些需求，因此容器的使用者不必擔心安裝任何項目。

這個容器映像檔將成為在 Cloud Run 上部署應用程式的基本建構區塊。由於容器幾乎可在任何虛擬或實體伺服器上使用，因此我們能將應用程式部署到您想要的任何位置，並將應用程式從一個服務供應商移至另一個服務供應商，或從地端移至雲端。

容器可讓應用程式：

- 可重現性 - 容器是獨立且完整的
- 可攜性：容器是跨產業的建構區塊，可讓應用程式在雲端供應商和環境之間攜帶

簡而言之，容器終於實現了「一次編寫，隨處執行」的目標。但這項規則有一個例外：產生的容器只能在您建立時使用的處理器類型上執行，不過您也可以為其他硬體設定產生容器版本。

說了這麼多，我們來建立容器吧！您將使用名為 **Docker** 的特定技術建立容器。

在 Cloud Editor 中，建立名為 `Dockerfile` 的新檔案。這個檔案是建構映像檔的藍圖。這項檔案會向 Docker 說明您的作業環境和原始碼、如何安裝依附元件、建構應用程式，以及執行程式碼。

```
# Use an official lightweight Python image.
FROM python:3.9-slim

# Copy local code to the container image.
WORKDIR /app
COPY main.py .
COPY requirements.txt .

# Install dependencies into this container so there's no need to
# install anything at container run time.
RUN pip install -r requirements.txt

# Service must listen to $PORT environment variable.
# This default value facilitates local development.
ENV PORT 8080

# Run the web service on container startup. Here you use the gunicorn
# server, with one worker process and 8 threads. For environments
# with multiple CPU cores, increase the number of workers to match
# the number of cores available.
CMD exec gunicorn --bind 0.0.0.0:$PORT --workers 1 --threads 8 --timeout 0 main:app
```

請詳閱內嵌註解 (「#」字元後的文字)，瞭解這段 `Dockerfile` 中各行的作用。

在 Cloud Terminal 中執行下列指令，使用 Cloud Build 建構容器映像檔：

```
gcloud builds submit --tag $TAG
```

Cloud Build 服務會建構容器映像檔，並將結果儲存在 Cloud Registry 中，方便您從網路上任何位置存取容器映像檔。

容器推送到 Artifact Registry 後，您會看到包含映像檔名稱的 `SUCCESS` 訊息，大致應如下所示：`gcr.io/<project-id>/hello`。映像檔現在會儲存在 Google Container Registry 中，您隨時隨地都能重複使用。

使用以下指令，列出與當前專案相關聯的所有容器映像檔：

```
gcloud container images list
```

現在請使用下列 `docker` 指令，透過 Cloud Shell 在本機執行及測試應用程式：

```
docker run -p $PORT:$PORT -e PORT=$PORT $TAG
```

如果 `docker` 指令無法提取遠端的容器映像檔，請嘗試執行以下指令：

```
gcloud auth configure-docker
```

`-p $PORT:$PORT` 選項會告知 Docker，將主機環境中的外部通訊埠 `$PORT` (上方設為 8080) 對應至執行中容器內的相同通訊埠號碼。這樣一來，您編寫的伺服器程式碼和測試應用程式時連線的外部通訊埠號碼會相同 (8080)，因此生活會更輕鬆。不過，您也可以使用 `-p` 選項，將主機上的任意外部通訊埠對應至容器內的任意內部通訊埠。

`-e PORT=$PORT` 選項會指示 Docker 將 `$PORT` 環境變數 (上方設為 8080) 提供給容器內執行的應用程式。

現在，您可以將網頁瀏覽器指向容器內執行的 Python 程式碼，測試應用程式。在 Cloud Shell 視窗中，按一下「網頁預覽」圖示，然後選取「透過以下通訊埠預覽：8080」，如同上一個步驟的操作。

結果應該很熟悉，您會看到隨機顏色背景前的生成文字，就像直接在 Cloud Shell 終端機中執行應用程式時一樣。重新載入頁面幾次，您會發現每次造訪應用程式時，背景顏色都會隨機變更。

在前往下一節之前，請在 Cloud Shell 終端機中輸入 `control-c` (按住 Control 鍵並按下「C」鍵)，停止執行中的應用程式。

恭喜！您現在已執行應用程式的容器化版本。在下一個部分中，您將容器映像檔轉換為正式品質的網頁應用程式，完全不必修改任何程式碼。

5. 雲端...



您已將應用程式容器化，現在要將這項成果分享給全世界，因此請將應用程式部署至雲端。但你希望的不只是分享，請務必確認：

- 穩定執行：如果執行應用程式的電腦當機，您會自動獲得容錯能力
- 自動調度資源：應用程式可因應大量流量，並在閒置時自動減少資源用量
- 盡量減少費用，因為系統不會對您未使用的資源收費，只會對處理流量時耗用的資源收費
- 可透過自訂網域名稱存取 - 您可以一鍵將自訂網域名稱指派給服務
- 提供優異的回應時間 - 冷啟動的回應速度合理，但您可以指定[執行個體數量下限](#)設定，進一步微調回應速度
- 使用標準 SSL/TLS 網路安全防護機制支援端對端加密 - 部署服務時，您會自動免費取得標準網路加密功能和相應的必要憑證

將應用程式部署至 Google Cloud Run，即可享有上述所有優點。

將應用程式部署至 Cloud Run

首先，請修改應用程式，以便區分新舊版本。為此，請修改 `main.py` 檔案，將預設訊息從「Hello world!」變更為「Hello from Cloud Run!」。換句話說，請將 `main.py` 中的這一行從以下內容變更為：

```
def say_hello(name="world"):
```

改成：

```
def say_hello(name="from Cloud Run"):
```

Cloud Run 具有「地區性」，這表示執行 Cloud Run 服務的基礎架構位於特定地區，並由 Google 代管，可為該地區內所有區域提供備援功能。在上述「開始設定」一節中，您已透過 `REGION` 環境變數定義預設區域。

如需目前支援的區域清單，請參閱「[Cloud Run \(全代管\) 地區](#)」。

執行下列指令，重新建構容器映像檔，並將容器化應用程式部署至 Cloud Run：

```
gcloud builds submit --tag $TAG
gcloud run deploy "$APP" \
  --image "$TAG" \
  --platform "managed" \
  --region "$REGION" \
  --allow-unauthenticated
```

- 您也可以使用 `gcloud config set run/region $REGION` 定義預設區域。
- `--allow-unauthenticated` 選項會將服務設為公開。如要避免未經驗證的要求，請改用 `--no-allow-unauthenticated`。

如要進一步瞭解可用選項，請輸入 `gcloud run deploy --help`。

這裡指定的映像檔是您在上一個步驟中建構的 Docker 映像檔。由於 Cloud Build 服務會將產生的映像檔儲存在 Google Container Registry 中，因此 Cloud Run 服務可以找到並為您部署該映像檔。

稍候片刻，等待部署作業完成。部署成功之後，指令列會顯示服務網址：

```
Deploying container to Cloud Run service [hello] in project [PROJECT_ID...]
✓ Deploying new service... Done.
  ✓ Creating Revision... Revision deployment finished. Waiting for health check...
  ✓ Routing traffic...
  ✓ Setting IAM Policy...
Done.
Service [hello] revision [hello-...] has been deployed and is serving 100 percent of
traffic.
Service URL: https://hello-....a.run.app
```

您也可以使用下列指令擷取服務網址：

```
gcloud run services describe hello \
  --platform managed \
  --region $REGION \
  --format "value(status.url)"
```

畫面應如下所示：

```
https://hello-....a.run.app
```

這個連結是 Cloud Run 服務專用的網址，並採用 TLS 安全性。這個連結是永久連結（只要您未停用服務），可在網路上任何位置使用。這項功能不會使用先前提及的 Cloud Shell Proxy 機制，該機制依附於暫時性虛擬機器。

按一下醒目的 `Service URL`，開啟網頁瀏覽器分頁，查看正在執行的應用程式。結果應該會顯示「Hello from Cloud Run!」訊息，背景則是隨機顏色。

恭喜！您的應用程式現已在 Google Cloud 中執行。您的應用程式會自動公開發布，並採用 TLS (HTTPS) 加密，還能自動擴充，因應驚人的流量。

但我認為這個程序可以更簡單...

6. 自動建立容器

這一切都很酷，但如果我不想考慮 Dockerfile 和容器，該怎麼辦？如果我像大多數開發人員一樣，只想專心編寫應用程式程式碼，讓其他人負責容器化作業，該怎麼做？好消息是，Cloud Run 支援名為 [Buildpacks](#) 的開放原始碼標準，這個標準的用途正是要自動化從一組來源檔案製造容器的程序。

請注意，在某些情況下，開發人員可能會偏好使用明確的 Dockerfile，例如希望高度自訂容器的建構方式。但對於這類常見情況，建構套件運作良好，可避免手動製作 `Dockerfile`。讓我們修改程式碼，以使用建構包。

首先，請修改應用程式，以便區分新舊版本。請修改 `main.py` 檔案，將預設訊息從「Hello from Cloud Run!」變更為「Hello from Cloud Run with Buildpacks!」。換句話說，請將 `main.py` 中的這一行從以下內容變更為：

```
def say_hello(name="from Cloud Run"):
```

改成：

```
def say_hello(name="from Cloud Run with Buildpacks"):
```

現在，我們來建立名為 `Procfile` 的新檔案，善用建構套件。請在 Cloud Editor 中建立該檔案，並插入這行文字：

```
web: python3 main.py
```

這會告知建構回溯系統如何在自動產生的容器中執行應用程式。有了這項指令，您甚至不需要 Dockerfile。如要驗證這點，請刪除 Dockerfile，並在 Cloud Shell 終端機中執行下列指令：

```
gcloud beta run deploy "$APP" \
  --source . \
  --platform "managed" \
  --region "$REGION" \
  --allow-unauthenticated
```

這與您在上一個步驟中執行的應用程式部署指令類似，但這次您已將 `--image` 選項替換為 `--source .` 選項。這會告知 `gcloud` 指令，您希望使用 Buildpacks 根據目前目錄中找到的來源檔案 (`--source .` 中的 `dot` 是目前目錄的簡寫) 建立容器映像檔。由於服務會隱含地處理容器映像檔，因此您不必在這個 `gcloud` 指令中指定映像檔。

再次點按醒目顯示的 `Service URL`，開啟網路瀏覽器分頁，確認部署作業是否成功，並確認服務是否在隨機顏色的背景上顯示「Hello from Cloud Run with Buildpacks!」。

請注意，使用建構套件製造 `Dockerfile` 時，您基本上已將三個簡單步驟簡化為兩個：

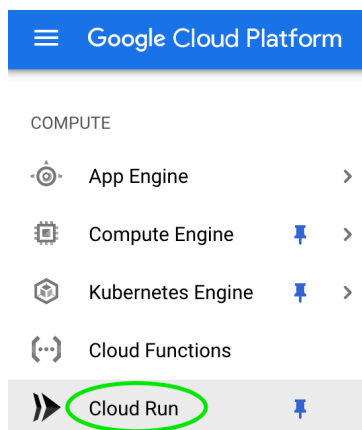
1. 在開發環境中建立應用程式。
 2. 透過單一指令，將完全相同的程式碼部署至雲端。
-

7. 我是否必須使用指令列？

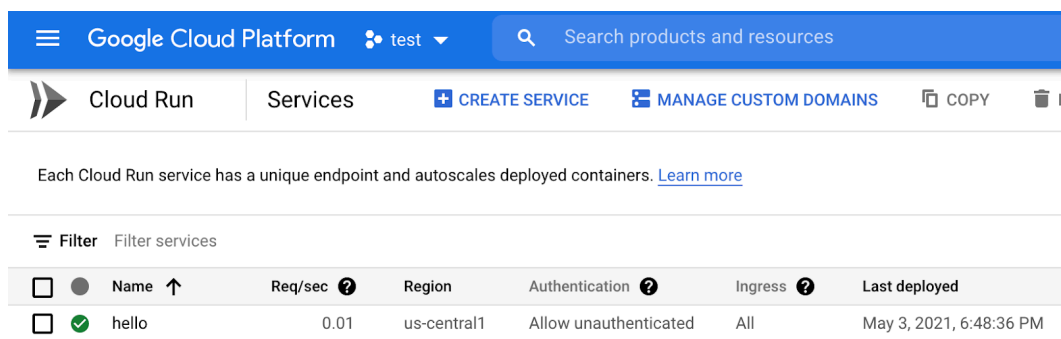
不需要。與幾乎所有 Google Cloud 服務一樣，您可透過三種方式與 Cloud Run 互動：

- gcloud 指令列工具 (您剛才已看過)。
- 透過 Cloud 控制台提供的豐富網頁使用者介面，支援直覺式的點選互動方式。
- 以程式設計方式，使用 Google 用戶端程式庫 (支援多種熱門語言，包括 Java、C#、Python、Go、Javascript、Ruby、C/C++ 等)。

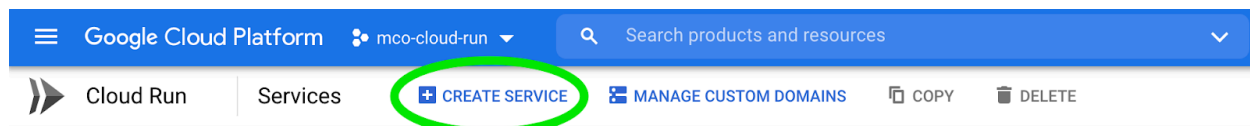
接下來，請使用控制台 UI 部署其他 Cloud Run 應用程式執行個體。透過左上方的選單前往 Cloud Run 服務到達網頁：



接著，您應該會看到 Cloud Run 服務的摘要，如下所示：



按一下「Create Service」連結，開始部署程序：



輸入「hello-again」做為服務名稱，採用預設的部署平台和區域，然後按一下「下一步」。

1 Service settings

A service exposes a unique endpoint and automatically scales the underlying infrastructure to handle incoming requests. Deployment platform and service name cannot be changed.

Service name *

hello-again

Deployment platform ?

☒ Cloud Run (fully managed)

Region *

us-central1 (Iowa)

[How to pick a region?](#)

☐ Cloud Run for Anthos

NEXT

輸入這個容器映像檔的網址：`gcr.io/cloudrun/hello`，這是 Google 為測試目的所建立的容器，然後按一下「進階設定」下拉式選單，即可查看許多可用的設定。以下列舉幾個可自訂的項目：

- 通訊埠號碼和容器進入點 (會覆寫建構容器時指定的進入點)
- 硬體：記憶體和 CPU 數量
- 資源調度：執行個體數量下限和上限
- 環境變數
- 其他：要求逾時設定、每個容器的要求數量上限、HTTP/2

按一下「下一步」按鈕，繼續進行對話。下一個對話方塊可讓您指定服務的觸發方式。在「Ingress」(輸入) 中選取「allow all traffic」(允許所有流量)，在「Authentication」(驗證) 中選取「Allow unauthenticated traffic」(允許未經驗證的流量)。

3 Configure how this service is triggered

A service can be invoked directly or via events. Click "Add Eventarc Trigger" to create a new event-based trigger. [Learn more](#)

Ingress ?

☒ Allow all traffic

☐ Allow internal traffic and traffic from Cloud Load Balancing

☐ Allow internal traffic only

Authentication * ?

☒ Allow unauthenticated invocations

Check this if you are creating a public API or website.

☐ Require authentication

Manage authorized users with Cloud IAM.

+ ADD EVENTARC TRIGGER

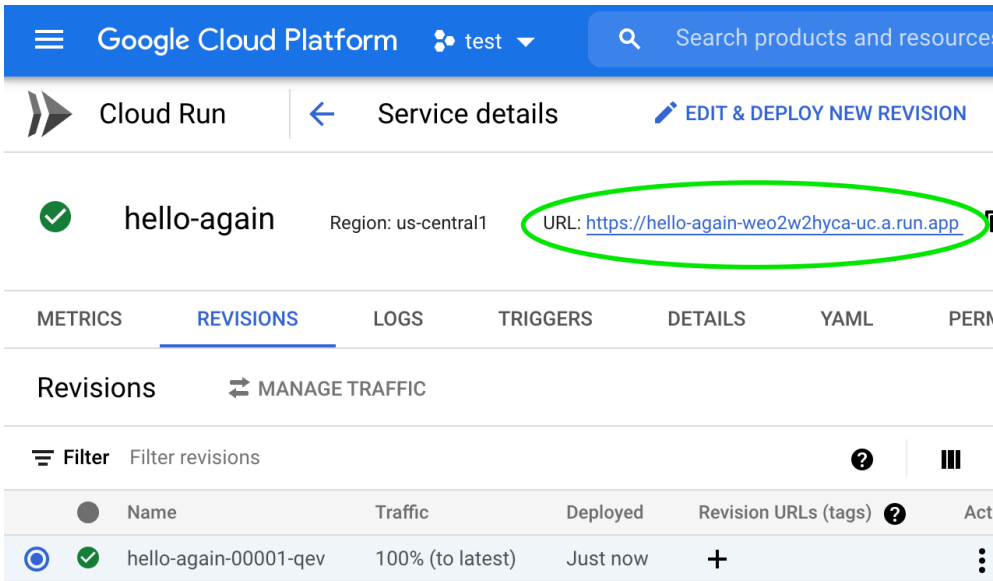
CREATE

CANCEL

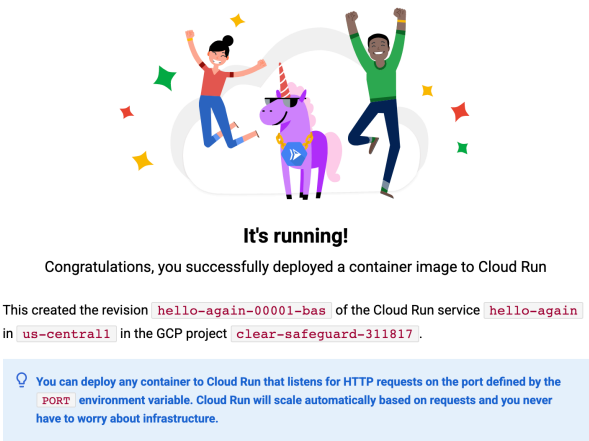
這是最寬鬆的設定，允許任何人從公用網際網路的任何位置存取 Cloud Run 應用程式，無須指定驗證憑證。您可能希望為應用程式設定更嚴格的限制，但為了方便學習，我們將簡化設定。

現在請按一下 `Create` 按鈕，建立 Cloud Run 服務。幾秒後，您應該會在 Cloud Run 服務的摘要清單中看到新服務。摘要行會顯示最近一次的部署作業 (日期/時間和部署者)，以及一些重要的設定。按一下服務名稱連結，即可深入瞭解新服務的詳細資料。

如要驗證服務，請按一下摘要頁面頂端顯示的網址，如下方範例所示：



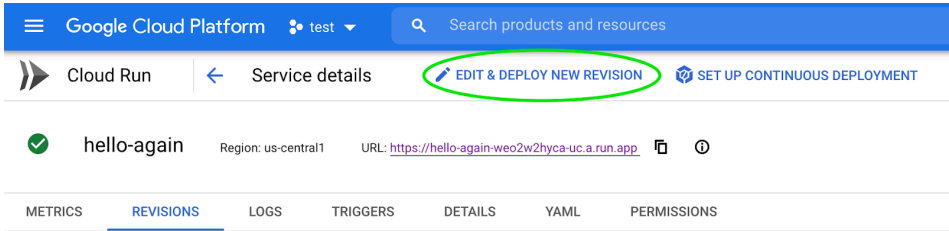
畫面應如下所示：



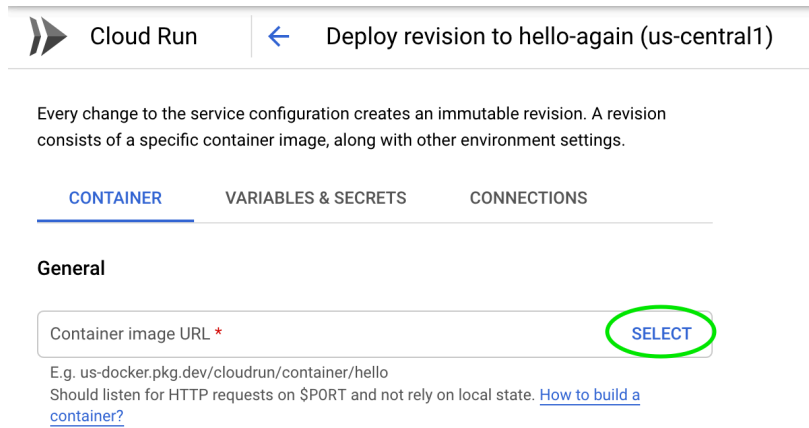
您已部署新的 Cloud Run 服務，現在請選取 REVISIONS 分頁，瞭解管理多個部署作業的方法。



如要直接從控制台部署新修訂版本，可以點選 EDIT & DEPLOY NEW REVISION 按鈕，如下方範例螢幕截圖中醒目顯示的部分：



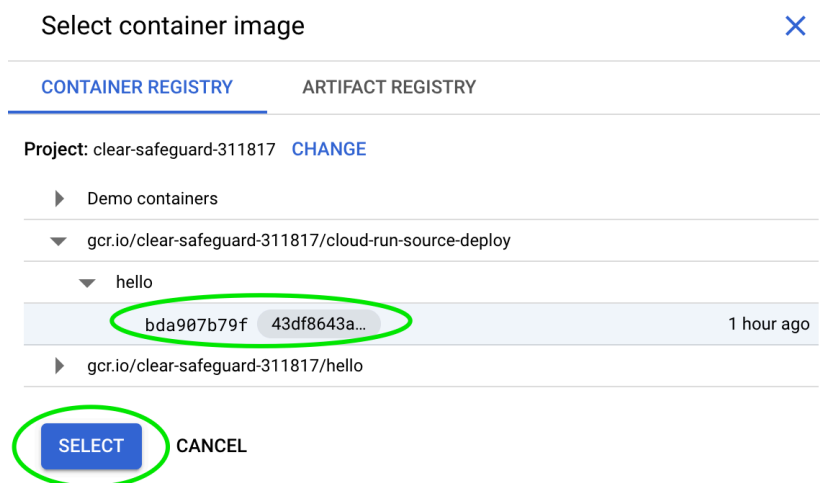
現在按一下該按鈕，即可建立新修訂版本。在容器網址旁，按一下 SELECT 按鈕，如下所示：



在彈出式對話方塊中，找出您先前使用 Buildpack 從 Cloud Build 部署的簡易型 Web 應用程式，然後按一下「選取」。請務必選擇「Container image」(容器映像檔) 下的

`gcr.io/<project>/cloud-run-source-deploy`

資料夾，如下所示：



選取檔案後，請捲動至底部，然後按一下 **DEPLOY** 按鈕。您已部署應用程式的新修訂版本。如要驗證，請再次前往服務網址，確認您現在看到的是色彩繽紛的「Hello from Cloud Run with Buildpacks!」網頁應用程式。

如您所見，「修訂版本」分頁會提供您部署的每個修訂版本摘要，現在您應該會看到這項服務有兩個修訂版本。如要選取特定修訂版本，請按一下修訂版本名稱左側的單選按鈕，畫面右側會顯示修訂版本詳細資料摘要。選取這些按鈕後，您會發現這兩個修訂版本是衍生自兩個不同的容器映像檔。

使用 **MANAGE TRAFFIC** 按鈕，即可修改傳送至特定修訂版本的傳入要求分配情形。這項功能可微調傳送至特定修訂版本的流量，因此有許多實用用途：

- 使用一小部分的連入流量，對新版應用程式進行初期測試
- 將流量從有問題的版本還原至先前的修訂版本
- A/B 測試

請根據您的裝置，瞭解如何找到 **MANAGE TRAFFIC** 按鈕：

METRICS	REVISIONS	LOGS	TRIGGERS	DETAILS	YAML	PER
Revisions MANAGE TRAFFIC						
Filter Filter revisions						
	Name	Traffic	Deployed	Revision URLs (tags)		
☒	hello-again-00002-yil	100% (to latest)	4 minutes ago	+		
☐	hello-again-00001-qev	0%	12 minutes ago			

如要將流量平分給兩個修訂版本，請指定 50/50 的流量分配，如下所示：

Manage traffic

Revision *	Traffic percentage
hello-again-00002-rif	50 %
hello-again-00001-bas	50 %
Latest healthy revision Serving	0 %

+ ADD REVISION

CANCEL **SAVE**

現在請按一下「儲存」按鈕，然後重複造訪服務的網址，確認要求平均有一半是由目前的修訂版本 (「Hello from Cloud Run with Buildpacks!」) 處理，另一半則是由先前的修訂版本 (「It's running!」) 處理，藉此驗證 50/50 的流量分配。

「服務詳細資料」頁面上的其他分頁標籤可監控效能、流量和記錄，有助於深入瞭解服務的運作負荷和成效。您也可以透過「權限」分頁微調服務存取權。請花幾分鐘瀏覽本頁面的分頁，瞭解可用的功能。

程式輔助介面

如先前所述，您也可以選擇以程式輔助方式建立、部署及管理 Cloud Run 服務。如果是手動工作，這個選項比指令列或網頁控制台更進階，但絕對是自動化 Cloud Run 服務的理想選擇。您可以選擇使用多種常見程式設計語言的 [Google 用戶端程式庫](#)。

步驟 8 · 約 6 分鐘

8. 測試應用程式



Photo by Biel Morro on Unsplash

最後一個步驟是執行人為負載測試，對應用程式進行壓力測試，並觀察應用程式如何因應需求變化而擴充。您將使用名為 [hey](#) 的工具，該工具已預先安裝在 Cloud Shell 中，可讓我們執行負載測試並呈現結果。

執行測試

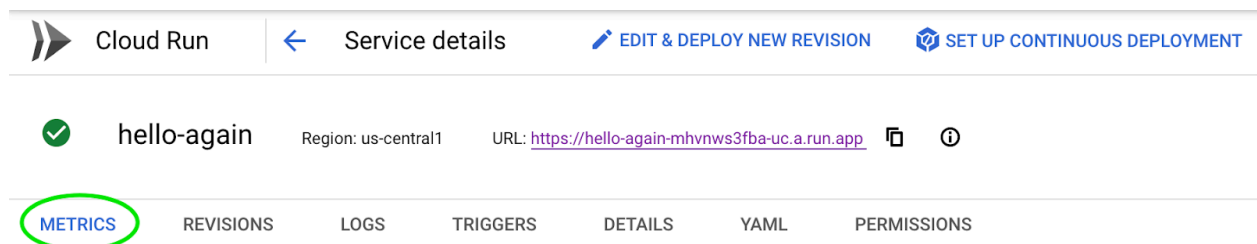
在 Cloud Shell 終端機中執行下列指令，進行負載測試：

```
hey -q 1000 -c 200 -z 30s https://hello-...run.app
```

指令引數的解讀方式如下：

- `-q 1000` - try to drive the load at roughly 1,000 requests per second
- `-c 200` - allocate 200 parallel workers
- `-z 30s` - 執行負載測試 30 秒
- 請務必使用服務網址做為這個指令列的最後一個引數

測試結果應如下所示：



步驟 9 · 約 1 分鐘

9. 清除

不使用服務時，Cloud Run 不會收費，但您可能仍須支付儲存所建構之容器映像檔的費用。

如要避免產生費用，您可以刪除 GCP 專案，這樣一來，系統就會停止對該專案中使用的所有資源計費。此外，也可以直接執行下列指令刪除容器映像檔：

```
gcloud container images delete $TAG
```

如要刪除 Cloud Run 服務，請使用下列指令：

```
gcloud run services delete hello --platform managed --region $REGION --quiet  
gcloud run services delete hello-again --platform managed --region $REGION --quiet
```

10. 你辦到了！



Photo by Mel Elías on Unsplash

恭喜！您已成功建構及部署正式版 Cloud Run 應用程式。過程中，您也瞭解了容器，以及如何建構自己的容器。您也瞭解到使用 `gcloud` 指令列工具和 Cloud Console，部署應用程式有多麼簡單。現在你已瞭解如何向全世界分享精彩創作！

最後，我想請您思考一個重要問題：

在開發人員環境中讓應用程式正常運作後，您必須修改多少程式碼，才能將應用程式部署至雲端，並享有 **Cloud Run** 提供的所有生產環境等級屬性？

答案當然是零。:)

如需其他語言的 Cloud Run 入門指南，請參閱以下網頁：<https://cloud.google.com/run/docs/quickstarts/build-and-deploy>。您可以在這裡找到 Go、Node.js、Java、C#、PHP、Ruby、Shell 指令碼和其他語言的操作說明。

程式碼研究室推薦...

- [在 Cloud Run 上使用 Node.js 建構 Slack 機器人](#)
- [透過 Eventarc 的事件觸發 Cloud Run](#)
- [Hello Cloud Run for Anthos Codelab](#)

- [使用 Node.js 在 Cloud Run 上部署及執行容器](#)
- [其他 Cloud Run 程式碼研究室](#)

其他值得探索的酷炫功能...

- [透過 Cloud Build 從 Git 持續部署](#)
- [使用 Secret Manager 中的密鑰](#)
- [Cloud Run 現已支援 WebSocket、HTTP/2 和 gRPC 雙向串流](#)
- [Cloud Run 執行個體數量下限：盡量減少無伺服器冷啟動次數](#)
- [提供來自多個區域的流量](#)

參考文件...

- [開發 Cloud Run 服務](#)
 - [建構容器](#)
 - [Cloud Run 說明文件](#)
 - [非官方 Cloud Run 常見問題](#)
 - [Cloud Run 定價](#)
-

11. 行動號召



如果您喜歡這個程式碼研究室，而且可能會花更多時間實際操作 **Google Cloud**，那麼請務必立即[加入 Google Cloud Innovators](#)！



Google Cloud Innovators 免費，並提供以下福利：

- 參加即時討論、AMA 和發展藍圖講座，直接取得來自 Google 員工的最新資訊。
- 直接在收件匣中掌握最新的 **Google Cloud** 消息
- 數位徽章和視訊會議背景
- 價值 \$500 美元的抵免額，用於 Skills Boost 的研究室和課程

按[這裡](#)報名！
